

1.1 Data type of all columns in the "customers" table.

SQL Query -

```
select
column_name,
data_type
from `sonic-fiber-493915-j6.Target.INFORMATION_SCHEMA.COLUMNS`
where table_name = 'customers';
```

SQL Response -

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

Business Response -

In the customers table, the columns customer_id, customer_unique_id, customer_city, customer_state have **STRING** datatype while customer_zip_code_prefix has **INT64** datatype.

1.2 Get the time range between which the orders were placed.

SQL Query -

```
select min(order_purchase_timestamp) as min_purchase_timestamp,
max(order_purchase_timestamp) as max_purchase_timestamp
from `Target.orders`;
```

SQL Response -

Row	min_purchase_timestamp	max_purchase_timestamp
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Business Response -

The time range between which the orders were placed is **2016-09-04 21:15:19 UTC** to **2018-10-17 17:30:18 UTC**

1.3 Count the Cities & States of customers who ordered during the given period.

SQL Query -

```
select
count(distinct c.customer_city) as city_count,
count(distinct c.customer_state) as state_count
from `Target.orders` o
left join
`Target.customers` c
on o.customer_id=c.customer_id
```

SQL Response -

Row	city_count	state_count
1	4119	27

Business Response -

The **city count** of the customers is **4119** and **state count** is **27** of customers who ordered during the given period. It suggest that out of 8011 distinct cities, the company serves customers in almost half of the total cities, indicating an opportunity for **geographic expansion**.

2.1 Is there a growing trend in the no. of orders placed over the past years?

SQL Query -

```
select extract(year from order_purchase_timestamp) as `year`,count(distinct order_id)
as No_of_orders_placed
from `Target.orders`
group by extract(year from order_purchase_timestamp)
order by `year`;
```

SQL Response -

Row	year	No_of_orders_placed
1	2016	329
2	2017	45101
3	2018	54011

Business Response -

Assuming the orders include orders with all possible order statuses, the orders **exponentially increased** from **2016 to 2018** indicating **increased customers, business growth and revenue**.

2.2 Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

SQL Query -

```
select format_date('%Y-%m',order_purchase_timestamp) as  
order_year_month,count(distinct order_id) as No_of_orders_placed  
from `Target.orders`  
group by format_date('%Y-%m',order_purchase_timestamp)  
order by order_year_month;
```

SQL Response -

Row	order_year_month	No_of_orders_placed
1	2016-09	4
2	2016-10	324
3	2016-12	1
4	2017-01	800
5	2017-02	1780
6	2017-03	2682
7	2017-04	2404
8	2017-05	3700
9	2017-06	3245
10	2017-07	4026

Business Response -

The orders **exponentially increased in 2017** with a **significant growth after November 2017**, the **orders peaked in November 2017, January 2018 and March 2018** respectively indicating **high discount sales, festive seasons etc.**

2.3 During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon
- 19-23 hrs : Night

SQL Query -

```
select
  case
    when extract(hour from order_purchase_timestamp) between 0 and 6 then 'Dawn'
    when extract(hour from order_purchase_timestamp) between 7 and 12 then 'Morning'
    when extract(hour from order_purchase_timestamp) between 13 and 18 then
'Afternoon'
    else 'Night'
  end as Order_Time,
  count(distinct order_id) as order_count
from `Target.orders`
group by order_time
order by order_count desc;
```

SQL Response -

Row	Order_Time	Order_Count
1	Afternoon	38135
2	Night	28331
3	Morning	27733
4	Dawn	5242

Business Response -

Most of the orders had been placed in the **Afternoon time**, followed by **Night** and **Morning Time** while a very **less no of orders** were placed in **Dawn Time**. Customers **mostly stay active on the platform during the Afternoon Time**.

3.1 Get the month on month no. of orders placed in each state.

SQL Query -

```
select
c.customer_state,
extract(year from o.order_purchase_timestamp) as Order_Year,
extract(month from o.order_purchase_timestamp) as Order_Month,
count(distinct o.order_id) as No_of_orders_placed
from `Target.orders` o
left join
`Target.customers` c
on o.customer_id=c.customer_id
group by customer_state, Order_Year, Order_Month
order by customer_state, Order_Year, Order_month;
```

SQL Response -

Row	customer_state	Order_Year	Order_Month	No_of_orders_pla...
1	AC	2017	1	2
2	AC	2017	2	3
3	AC	2017	3	2
4	AC	2017	4	5
5	AC	2017	5	8
6	AC	2017	6	4
7	AC	2017	7	5
8	AC	2017	8	4
9	AC	2017	9	5
10	AC	2017	10	6

Business Response -

High monthly orders indicate **increased customer interest or demand** while **low orders** in a particular **region** indicate inefficiencies like **high delivery charges, damaged product or poor reviews** etc.

3.2 How are the customers distributed across all the states?

SQL Query -

```
select customer_state,  
count(distinct customer_unique_id) as No_of_customers  
from `Target.customers`  
group by customer_state  
order by No_of_customers desc;
```

SQL Response -

Row	customer_state	No_of_customers
1	SP	40302
2	RJ	12384
3	MG	11259
4	RS	5277
5	PR	4882
6	SC	3534
7	BA	3277
8	DF	2075
9	ES	1964
10	GO	1952

Business Response -

SP, RJ, MG have the most no of customers with **SP** being the highest, having count of **40,302** customers indicating strong market presence while **RR** being the lowest with just **45** customers.

4.1 **Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).**

SQL Query -

```
with Order_cost as (  
    select  
        extract(year from o.order_purchase_timestamp) as `Order_year`,  
        sum(p.payment_value) as Total_order_cost  
    from `Target.orders` o  
    join  
        `Target.payments` p  
    on o.order_id=p.order_id  
    where extract(year from o.order_purchase_timestamp) in (2017, 2018) and  
        extract(month from o.order_purchase_timestamp) in (1,2,3,4,5,6,7,8)  
    group by Order_year  
    order by Order_year  
)  
  
select (  
    (select Total_order_cost from order_cost where Order_year=2018)  
    -  
    (select Total_order_cost from order_cost where Order_year=2017)  
    ) * 100 / (select Total_order_cost from order_cost where Order_year=2017)  
    as Percentage_increase  
from Order_cost;
```

SQL Response -

Row	Percentage_increase
1	136.97687164667107
2	136.97687164667107

Business Response -

The **Total Order Cost** increased by **136.97%** from the year **2017** to **2018** indicating significant rise in business growth and customer purchases.

4.2 Calculate the Total & Average value of order price for each state.

SQL Query -

```
select
c.customer_state,
sum(i.price) as Total_price,
avg(i.price) as Avg_price
from `Target.customers` c
left join
`Target.orders` o
on c.customer_id=o.customer_id
join
`Target.order_items` i
on i.order_id=o.order_id
group by c.customer_state;
```

SQL Response -

Row	customer_state	Total_price	Avg_price
1	AC	15982.949999999999	173.72771739130428
2	AL	80314.809999999954	180.88921171171174
3	AM	22356.840000000026	135.49600000000004
4	AP	13474.299999999992	164.32073170731704
5	BA	511349.99000000587	134.6012082126874
6	CE	227254.70999999752	153.75826116373523
7	DF	302603.93999999779	125.77054862842888
8	ES	275037.30999999656	121.91370124113485
9	GO	294591.9499999964	126.2717316759535
10	MA	119648.21999999999	145.20415048543703

Business Response -

The states with **higher total order price** has a **significant contribution** towards **company revenue** while states with **higher average order price** indicate customers purchasing **expensive items**.

4.3 Calculate the Total & Average value of order freight for each state.

SQL Query -

```
with state_freight_price as (  
    select  
        o.order_id,  
        c.customer_state,  
        i.freight_value  
    from `Target.customers` c  
    left join  
        `Target.orders` o  
    on c.customer_id=o.customer_id  
    join  
        `Target.order_items` i  
    on i.order_id=o.order_id  
)  
  
select customer_state,  
sum(freight_value) as Total_freight_value,  
avg(freight_value) as Avg_freight_value  
from state_freight_price  
group by customer_state;
```

SQL Response -

Row	customer_state	Total_freight_value	Avg_freight_value
1	AC	3686.749999999...	40.07336956521...
2	AL	15914.589999999...	35.84367117117...
3	AM	5478.89	33.20539393939...
4	AP	2788.500000000...	34.00609756097...
5	BA	100156.6799999...	26.36395893656...
6	CE	48351.589999999...	32.71420162381...
7	DF	50625.499999999...	21.04135494596...
8	ES	49764.599999999...	22.05877659574...
9	GO	53114.979999999...	22.76681525932...
10	MA	31523.770000000...	38.25700242718...

Business Response -

The states with **higher total freight values** indicate high no of products delivered in that particular region while states with **higher average freight price** may indicate that the **products are heavy** or the travelling **distance is too long**.

5.1 Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

SQL Query -

```
select
distinct order_id,
date_diff(order_delivered_customer_date,order_purchase_timestamp,day) as
delivery_time,
date_diff(order_delivered_customer_date,order_estimated_delivery_date,day) as
diff_estimated_delivery
from `Target.orders`
where order_status not in ('canceled','unavailable')
and order_delivered_customer_date is not null
and order_estimated_delivery_date is not null ;
```

SQL Response -

Row	order_id	delivery_time	diff_estimated_d...
1	bfb0f9bdef84302105ad712db...	54	36
2	98974b076b01553d49ee64679...	43	-6
3	c4b41c36dd589e901f6879f25a...	36	-14
4	d2292ff2201e74c5db154d1b7a...	29	-20
5	95e01270fcb9e986342340010...	30	-19
6	ed8c7b1b3eb256c70ce0c7423...	44	-5
7	5cc475c7c03290048eb2e742c...	68	18
8	6b3ee7697a02619a0ace2b3f0a...	47	-2
9	3b2ca3293a7ce539ea2379d70...	43	-7
10	b2f92b2f7047cd8b35580d629d...	43	-7

Business Response -

Orders with **high delivery time** may indicate **long shipping distance and time**, **unavailability of warehouses** to store products while **high positive difference between the estimated & actual delivery date** indicates **operational inefficiency**.

5.2 Find out the top 5 states with the highest & lowest average freight value.

SQL Query -

```
with state_freight_price as (
    select
        o.order_id,
        c.customer_state,
        i.freight_value
    from `Target.customers` c
    left join
        `Target.orders` o
    on c.customer_id=o.customer_id
    join
        `Target.order_items` i
    on i.order_id=o.order_id
)
```

```

, top_5 as (
  select customer_state, avg(freight_value) as avg_freight_value from
state_freight_price
  group by customer_state
  order by avg(freight_value) desc
  limit 5
)
, bottom_5 as (
  select customer_state, avg(freight_value) as avg_freight_value from
state_freight_price
  group by customer_state
  order by avg(freight_value)
  limit 5
)

select customer_state, avg_freight_value, 'Top_5' as category from top_5
union all
select customer_state, avg_freight_value, 'Bottom_5' as category from bottom_5;

```

SQL Response -

Row	customer_state	avg_freight_value	category
1	RR	42.984423076923079	Top_5
2	PB	42.723803986710934	Top_5
3	RO	41.069712230215828	Top_5
4	AC	40.073369565217391	Top_5
5	PI	39.147970479704782	Top_5
6	SP	15.147275390419287	Bottom_5
7	PR	20.531651567944341	Bottom_5
8	MG	20.630166806306551	Bottom_5
9	RJ	20.9609239316825	Bottom_5
10	DF	21.041354945968447	Bottom_5

Business Response -

RR, PB, RO, AC, PI have **higher average freight price** which may indicate that the **products are fragile, heavy** or the travelling **distance is too long** or operational inefficiency while **SP, PR, MG, RJ, DF** have **lower average freight price** which may indicate that the **products are lightweight**, or the **transportation expenses are low**.

5.3 Find out the top 5 states with the highest & lowest average delivery time.

SQL Query -

```
with order_delivery_time as (  
    select  
        customer_id, order_id,  
        date_diff(order_delivered_customer_date,order_purchase_timestamp,day) as  
delivery_time  
    from `Target.orders`  
    where order_status not in ('canceled','unavailable')  
    and order_delivered_customer_date is not null  
)  
, state_delivery_time as (  
    select c.customer_state,  
        t.delivery_time  
    from order_delivery_time t  
    left join  
        `Target.customers` c  
    on t.customer_id=c.customer_id  
)  
  
(select customer_state,  
avg(delivery_time) as avg_delivery_time,  
'Highest_avg_delivery_time' as Category  
from state_delivery_time  
group by customer_state  
order by avg_delivery_time desc limit 5)  
union all  
(select customer_state,  
avg(delivery_time) as avg_delivery_time,  
'Lowest_avg_delivery_time' as Category  
from state_delivery_time  
group by customer_state  
order by avg_delivery_time limit 5);
```

SQL Response -

Row	customer_state	avg_delivery_time	Category
1	RR	28.97560975609...	Highest_avg_delivery_time
2	AP	26.73134328358...	Highest_avg_delivery_time
3	AM	25.98620689655...	Highest_avg_delivery_time
4	AL	24.04030226700...	Highest_avg_delivery_time
5	PA	23.31606765327...	Highest_avg_delivery_time
6	SP	8.298093544722...	Lowest_avg_delivery_time
7	PR	11.52671135486...	Lowest_avg_delivery_time
8	MG	11.54218777523...	Lowest_avg_delivery_time
9	DF	12.50913461538...	Lowest_avg_delivery_time
10	SC	14.47518330513...	Lowest_avg_delivery_time

Business Response -

RR, AP, AM, AL, PA have **highest average delivery time** which may indicate **unavailability of warehouses, slow logistics transportation , courier challenges or the travelling distance is too long** while **SP, PR, MG, DF, SC** have **lowest average delivery time** which may indicate **high operational efficiency** and **customer satisfaction**.

5.4 Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

SQL Query -

```
with order_delivery_time as (  
  select  
    customer_id, order_id,  
    order_delivered_customer_date,  
    order_estimated_delivery_date  
  from `Target.orders`  
  where order_status not in ('canceled','unavailable')  
  and order_delivered_customer_date is not null  
  and order_estimated_delivery_date is not null  
)
```

```

, state_delivery_delay_time as (
  select c.customer_state,
  date_diff(order_estimated_delivery_date,order_delivered_customer_date,day) as
delay_time
  from order_delivery_time t
  left join
  `Target.customers` c
  on t.customer_id=c.customer_id
)

select customer_state,
avg(delay_time) as avg_delay_time
from state_delivery_delay_time
group by customer_state
order by avg_delay_time desc;

```

SQL Response -

Row	customer_state	avg_delay_time
1	AC	19.762499999999996
2	RO	19.131687242798357
3	AP	18.731343283582092
4	AM	18.606896551724137
5	RR	16.414634146341459
6	MT	13.431151241534995
7	PA	13.190274841437626
8	RS	12.981848802395183
9	RN	12.757383966244728
10	PE	12.401757689893268

Business Response -

States with **high positive average delivery delay time** indicate **high demand, happy customers, excellent logistics performance, efficient operations etc.**

6.1 Find the month on month no. of orders placed using different payment types.

SQL Query -

```
with customer_payment_type as (  
    select o.order_id,  
           format_timestamp('%Y-%m',o.order_purchase_timestamp) as purchase_year_month,  
    p.payment_type  
    from `Target.orders` o  
    join  
    `Target.payments` p  
    on o.order_id=p.order_id  
)  
  
select payment_type,  
       purchase_year_month,  
       count(distinct order_id) as No_of_Orders_Placed  
from customer_payment_type  
group by payment_type, purchase_year_month  
order by lower(payment_type), purchase_year_month;
```

SQL Response -

Row	payment_type	purchase_year_month	No_of_Orders_Placed
1	credit_card	2016-09	3
2	credit_card	2016-10	253
3	credit_card	2016-12	1
4	credit_card	2017-01	582
5	credit_card	2017-02	1347
6	credit_card	2017-03	2008
7	credit_card	2017-04	1835
8	credit_card	2017-05	2833
9	credit_card	2017-06	2452
10	credit_card	2017-07	3072

Business Response -

It is observed that **most of the orders** have been made through **credit card**, while a very **less no of orders** are made using other payment types like **UPI, Debit card or voucher**.

6.2 Find the no. of orders placed on the basis of the payment installments that have been paid.

SQL Query -

```
select payment_installments,  
count(distinct order_id) as No_of_Orders  
from `Target.payments`  
group by payment_installments  
order by No_of_orders desc;
```

SQL Response -

Row	payment_installments	No_of_Orders
1	1	49060
2	2	12389
3	3	10443
4	4	7088
5	10	5315
6	5	5234
7	8	4253
8	6	3916
9	7	1623
10	9	644

Business Response -

Most number of orders have been paid in 1 installment only, followed by 2 and 3 installments.